TP2 AED2 – Justificación de Complejidades

Grupo mandarinaConcordia

Apellido y Nombre	**LU**	**Mail**
Correa Cravo, Enzo	1079/25	enzoccravo06@gmail.com
Ambricca, Marco	1011/25	mambricca@dc.uba.ar
Mantiñan, Joaquin	1132/25	mantodg6@gmail.com
Cossi, Juan Pablo	1037/25	cossijuanpablo@gmail.com

Estructuras que usamos

Sea U la cantidad de usuarios y |L| el largo del nombre de una promoción.

- **DiccionarioABB de usuarios** (clave: ID, valor: el handle del usuario en el heap). Buscar e insertar cuesta O(log U) en promedio y O(U) en el peor caso, cuando el árbol queda desbalanceado. Como guardamos el handle, con una sola búsqueda tenemos el usuario y su referencia en el heap.
- **Max-heap por millas disponibles**, para el ranking. Encolar, desencolar y reacomodar un elemento cuestan O(log U).
- **Trie de promociones**. Insertar, buscar y borrar cuestan O(|L|).
- **masAcumulador y masCanjeador** como atributos del sistema.

Complejidad de cada operación

crearSistema - O(1). Crea las estructuras vacías. También carga la promoción "NoPromo" (factor 1, sin tope), pero su nombre tiene largo fijo, así que sigue siendo O(1).

registrarUsuario / registrarUsuarioConReferidor - O(log U) promedio, O(U) peor caso. Encolamos al usuario en el heap (O(log U)) e insertamos su handle en el ABB (O(log U) promedio, O(U) peor caso). Con referidor, además buscamos a los dos usuarios en el ABB, les sumamos 500 millas en O(1) y los reacomodamos en el heap en O(log U).

acumularMillas - O(log U + |L|) promedio, O(U + |L|) peor caso. Buscamos la promoción en el trie (O(|L|)); si no existe, usamos "NoPromo". Buscamos al usuario en el ABB (O(log U) promedio, O(U) peor caso), calculamos las millas con su categoría y el tope en O(1), lo comparamos con masAcumulador en O(1) y lo reacomodamos en el heap en O(log U).

canjearMillas - O(log U) promedio, O(U) peor caso. Buscamos al usuario en el ABB, descontamos las millas en O(1), comparamos con masCanjeador en O(1) y reacomodamos el heap en O(log U).

transferirMillas - O(log U) promedio, O(U) peor caso. Buscamos origen y destino en el ABB, movemos las millas en O(1) y reacomodamos a los dos en el heap (O(log U) cada uno). También actualizamos masCanjeador y los datos de exclusividad, todo en O(1).

consultarSaldo / consultarCategoria - O(log U) promedio, O(U) peor caso. Buscamos al usuario en el ABB y devolvemos el saldo, o calculamos la categoría comparando sus millas con las categorias posibles.

iniciarPromocion / iniciarPromocionConTope - O(|L|). Creamos la promoción y la insertamos en el trie con su nombre como clave. La información de la promoción queda en el nodo de la última letra.

finalizarPromocion - O(|L|). Borramos la promoción del trie. A partir de ahí, acumularMillas usa factor 1.

elMasAcumulador / elMasCanjeador - O(1). Los tenemos como atributos y los actualizamos cada vez que alguien acumula o canjea, así que solo hay que devolverlos.

rankingMillas - O(N log U). Sacamos N veces el máximo del heap (O(log U) cada vez) y después volvemos a encolar a esos N usuarios para no perder el ranking. Son 2*n operaciones de O(log U), osea, O(N log U).

paresTransferidoresExclusivos - O(U). Cada usuario guarda con quién operó en la flag exclusivo y si esa relación sigue siendo exclusiva en la flag esExclusivo. Esos datos se actualizan en O(1) en cada transferencia. Para armar el res, recorremos el ABB en orden (O(U) en total) y, para cada usuario, revisamos en O(1) si él y su exclusivo siguen siendo exclusivos. Para no repetir pares, guardamos el par solo desde el usuario de menor ID.